



Retrieval-Based Residual Adaptation

A lightweight proposal for adapting pretrained time-series forecasters

Gaspard Berthelier

Neo Seminar – June 30, 2026

EDF R&D / Inria Sophia Antipolis

My research

Proposal Focus

RESEARCH DIRECTION

Time Series Forecasting on Heterogeneous Data

Supervising team

Academic (Inria): Giovanni Neglia

Industrial (EDF): Tahar Nabil, Etienne Le Naour, Richard Niamke

Start date

2nd February 2025

Proposal Focus

RESEARCH DIRECTION

Time Series Forecasting on Heterogeneous Data

Supervising team

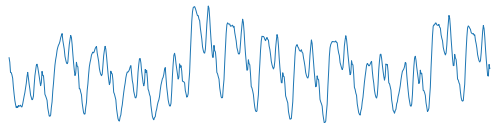
Academic (Inria): Giovanni Neglia

Industrial (EDF): Tahar Nabil, Etienne Le Naour, Richard Niamke

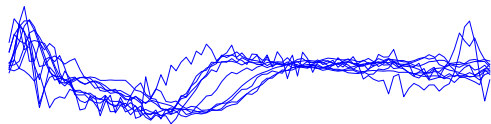
Start date

2nd February 2025

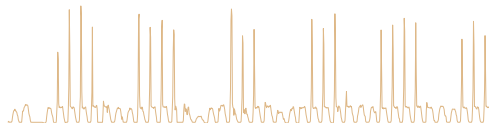
Time Series (TS)



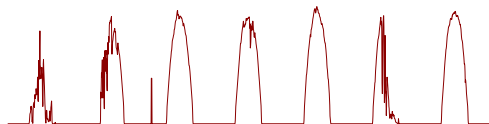
Aggregated load curve



Heartbeat electrical activity



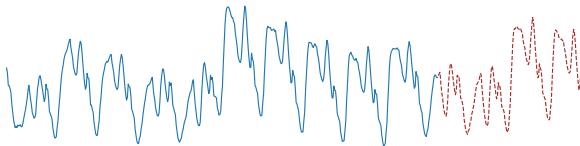
Road occupancy



Solar power generation

Time Series Forecasting (TSF)

Given a lookback window $\mathbf{x} = (\mathbf{x}_{t-L+1}, \dots, \mathbf{x}_t) \in \mathbb{R}^L$, time-series forecasting aims to predict a future horizon $\mathbf{y} = (\mathbf{x}_{t+1}, \dots, \mathbf{x}_{t+H}) \in \mathbb{R}^H$.



Individual electricity load curve: observed lookback and horizon to forecast.

Current context

Foundation Models (FMs)

Foundation models, such as Chronos-2, TabPFN-TS, and TS-ICL (???), have become state-of-the-art in forecasting. Notable properties:

- Pretrained on huge datasets mixing real-world and synthetic time series (large **prior**).
- Perform gradient-free **in-context learning**
- Modern architectures take optional **covariates** as inputs:

$$\hat{y} = f_{\theta}(x, z) \in \mathbb{R}^{L+H}$$

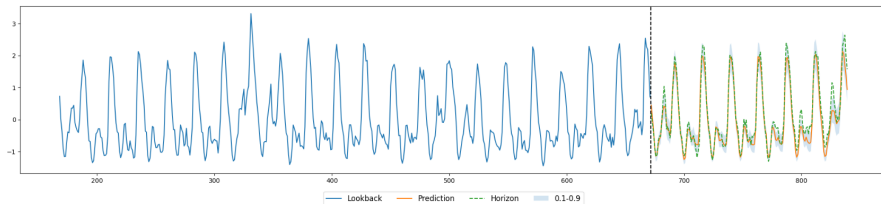


Figure 1: Example prediction of Chronos-2 on ELECTRICITY

How to Adapt to Local Context?

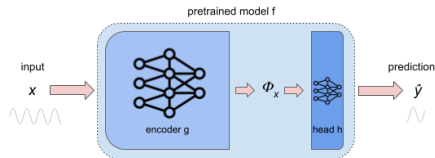


Figure 2: Illustration of a pretrained forecasting model. Models such as **Chronos-2** (120M parameters) are composed of multiple temporal and cross-variate attention layers.

- **Fine-tuning:** update part or all of the pretrained model on local data.
- **LoRA:** learn low-rank adaptation weights while keeping most pretrained parameters fixed.
- **Adding covariates:** adding *relevant* covariates to the inputs.
Promising direction to adapt locally *without intensive tuning*

Adding covariates is not trivial

Assume $Y_t = f^*(t, Z_t) = a + bt + Z_t$ with $Z_t = ct$.

- Optimal linear model on t is $Y_t = f_0(t) = a + (b + c)t$
- One possible model over t and Z_t is $Y_t = f_2(t, Z_t) = a + 0.t + \frac{b+c}{c}Z_t$

Now if $Z_t \sim Z_{t-1} + \epsilon_t$, its information is crucial.

This examples shows adding information can both be necessary or detrimental (even when causally relevant).

Finding *relevant* covariates for prediction is essential. But what is relevant? In our setting, we only have access to one's own past data, and other users' past data.

Adding covariates is not trivial

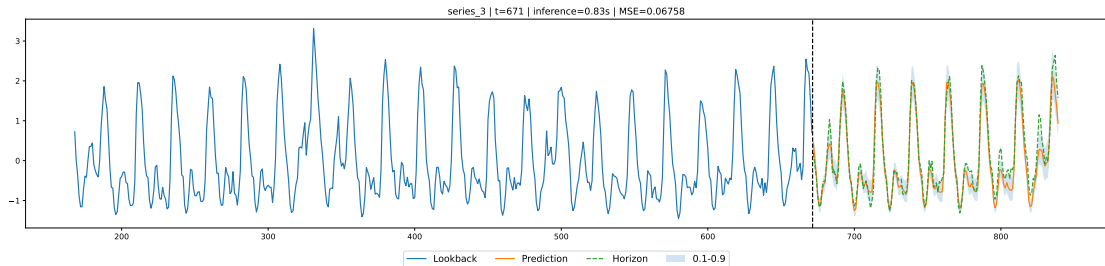


Figure 3: Example *univariate* prediction, MSE=0.068

Adding covariates is not trivial

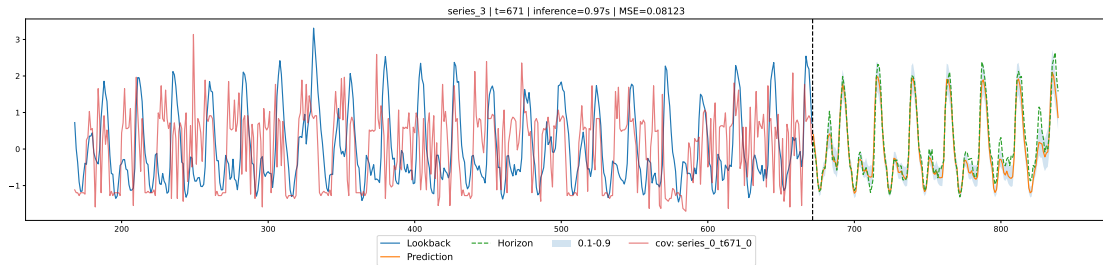


Figure 3: Example prediction with covariate, MSE=0.2293

Adding covariates is not trivial

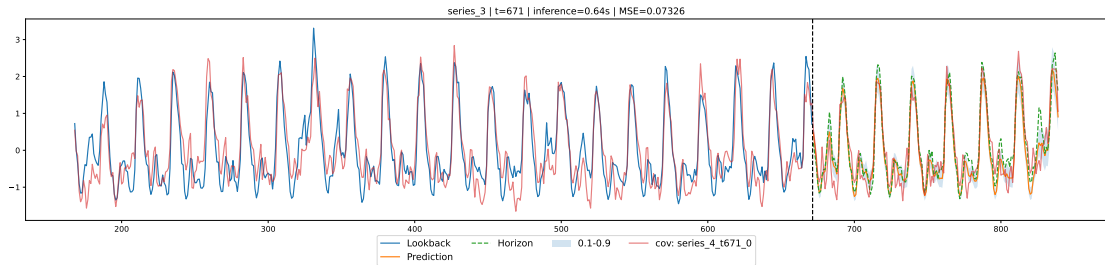


Figure 3: Example prediction with covariate, MSE=0.0733

Adding covariates is not trivial

- Current FMs do not detect easy causal cross-variate patterns very well, even if all information is provided in context (?)
- We tested adding closest windows as covariates: on average, it degrades performance.

- On roughly 50% of windows, closest windows do improve performance.
With optimal decision, can achieve 10% improvement!

Idea: learn a gate that decides when to use $\hat{y} = f(\mathbf{x})$ or $\hat{y}^c = f(\mathbf{x}, \mathbf{z})$.

Gated conditioned prediction

We'd want to train a model:

$$y_{adapted} = \sigma(\hat{y}, \hat{y}^c)$$

$$y_{adapted} = \sigma(x, X_c, \hat{y}, \hat{y}^c)$$

Mettre tableau benchmark des 2*2*2 versions (scalar, horizon), avec et sans features, avec et sans instance norm pour la distance, avec 1, 3 10 voisins. Mettre slide annexe avec features choisis. Ajouter chapeaux sur y.

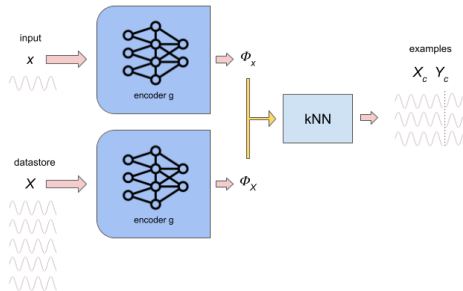
Retrieval-Augmented Generation (RAG)

For a target lookback window $\mathbf{x}$, retrieve aligned historical neighbors from the local datastore:

$$\mathcal{N}(\mathbf{x}) = \{(\mathbf{x}_k, \mathbf{y}_k)\}_{k=1}^K, \quad \mathbf{x}_k \approx \mathbf{x}.$$

This allows to compute the context-conditioned forecast $\hat{\mathbf{y}}^c = f(\mathbf{x}, \mathcal{N}(\mathbf{x}))$.

But it also gives us past ground-truths $\{\mathbf{y}_k\}_{k=1}^K$

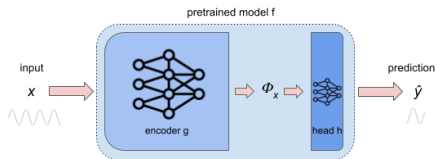


k-NN retrieval of aligned historical examples.

Fetching neighbors

- Fitting a convex combination of outputs with local datastore is beneficial in NLP tasks (??).
- Recent models such as Cross-RAG (?) fuse retrieved examples' *latent representations* with the query, to decode a new forecast.

It is possible to learn to fuse information from neighbors to improve prediction!



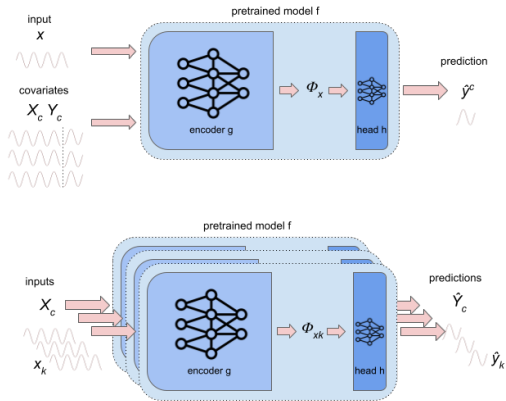
Proposal

Retrieval-Based Residual Adaptation

The adapter receives four complementary signals:

- the vanilla forecast $\hat{y}$;
- the context-conditioned forecast $\hat{y}^c$;
- realized horizons Y_c from similar examples;
- local residuals $R_c = Y_c - \hat{Y}_c$.

The goal is to improve $\hat{y}$ while keeping the correction lightweight, interpretable, and conservative.



Extra predictions expose both memory and model errors.

Baseline Adapters

Before introducing a deeper adapter, compare against simple output-space corrections:

Adapter	Signal used	Trainable part
Gate on conditioned-prediction	$\hat{y}, \hat{y}^c$	scalar or horizon-wise decisions
Weighted mix of horizons	$\hat{y}, Y_c$	fixed via distance, learned convex or ridge
Weighted residual	$\hat{y}, R_c$	fixed via distance, learned convex or ridge
Full mixture	$\hat{y}, \hat{y}^c, Y_c, R_c$	scalar or per-horizon

Example formulation of fixed mixture with one learned coefficient:

$$\hat{y}_{\text{mix}} = (1 - \lambda)\hat{y} + \lambda \sum_{k=1}^K w_k y_k, \quad w_k = \frac{\exp(-d_k)}{\sum_{j=1}^K \exp(-d_j)}.$$

Complete the following tables after the baseline sweep.

Baseline Sweep Results

Dataset	Retrieval	Context	kNN	Residual	r-mix0	r-mix1	r-mix2
Electricity 168-24	raw, $K = 1$	–	–	–	–	–	–
Electricity 168-24	raw, $K = 3$	–	–	–	–	–	–
Electricity 168-24	raw, $K = 10$	–	–	–	–	–	–
Electricity 168-24	IN, $K = 1$	–	–	–	–	–	–
Electricity 168-24	IN, $K = 3$	–	–	–	–	–	–
Electricity 168-24	IN, $K = 10$	–	–	–	–	–	–

nMSE on T3. Repeat the same table for 672-168 and for Solar.

Gate Sweep Results

Dataset	Retrieval	Cls-s	Cls-h	Reg-s	Reg-h	Oracle-s	Oracle-h
Electricity 168-24	raw, $K = 1$	-	-	-	-	-	-
Electricity 168-24	raw, $K = 3$	-	-	-	-	-	-
Electricity 168-24	raw, $K = 10$	-	-	-	-	-	-
Electricity 168-24	IN, $K = 1$	-	-	-	-	-	-
Electricity 168-24	IN, $K = 3$	-	-	-	-	-	-
Electricity 168-24	IN, $K = 10$	-	-	-	-	-	-

CatBoost gates are kept as the decision baselines.

Residual and Memory Branches

Define joint input-prediction representations

$$m = [x, \hat{y}], \quad M_c = [X_c, \hat{Y}_c].$$

RESIDUAL BRANCH

$$z_r = \text{Attn}(m, M_c, R_c), \quad y_r = \hat{y} + \text{MLP}(\hat{y}, z_r).$$

z_r asks how the frozen model was wrong in similar states.

MEMORY BRANCH

$$z_m = \text{Attn}(x, X_c, Y_c), \quad y_m = \hat{y} + \text{MLP}([\hat{y}, z_m]).$$

z_m learns how much realized neighbor horizons inform on the forecast.

Final Mixture and Training Loss

The final layer mixes the four candidate forecasts:

$$\tilde{Y} = [\hat{y}, \hat{y}^c, y_r, y_m] \in \mathbb{R}^{4 \times H}, \quad z = \text{Attn}(x, X_c, X_c) \in \mathbb{R}^{1 \times L}$$

$$\alpha = \text{softmax}(\text{MLP}(z)) \in \mathbb{R}^{4 \times H} \quad \hat{y}_{\text{adapted}} = \alpha \odot \tilde{Y} = \left(\sum_{i=1}^4 \alpha_{i,h} \tilde{Y}_{i,h} \right)_h$$

TS-IFA Results

Dataset	<i>L-H</i>	Chronos	Best baseline	Best gate	TS-IFA
Electricity	168-24	–	–	–	–
Electricity	672-168	–	–	–	–
Solar	168-24	–	–	–	–
Solar	672-168	–	–	–	–

nMSE on T3. Lower is better.

Thank you
Questions?

Appendix

Appendix: Baseline Formulas

Distance weights:

$$w_k = \frac{\exp(-d_k)}{\sum_j \exp(-d_j)}.$$

Fixed horizon and residual corrections:

$$\hat{y}_{\text{kNN}} = \sum_k w_k y_k, \quad \hat{y}_{\text{res}} = \hat{y} + \sum_k w_k r_k.$$

Learned residual mixes:

$$\hat{y}_{\text{r0}} = \hat{y} + \lambda \sum_k w_k r_k, \quad \hat{y}_{\text{r1}} = \hat{y} + \sum_k \beta_k r_k, \quad \hat{y}_{\text{r2},h} = \hat{y}_h + \sum_k \beta_{k,h} r_{k,h}.$$

Appendix: Gate Features

Gate targets compare vanilla and context-conditioned losses:

$$\Delta_h = (y_h - \hat{y}_h)^2 - (y_h - \hat{y}_h^c)^2.$$

- Scalar gates use horizon mean and standard deviation of $\hat{y}^c - \hat{y}$.
- Horizon gates use the full vector $\hat{y}^c - \hat{y}$.
- Both use neighbor residual summaries, query and neighbor moments, same-user ratio, neighbor age, weight concentration, and mean distance.

Appendix: Experiment Grid

Axis	Values	Role
Retrieval normalization	raw, IN	without or with instance normalization
Neighbors	1, 3, 10	local memory size
Fit split	T1	mixture baselines and TS-IFA
Gate split	T2	CatBoost decision baselines
Evaluation split	T3	final untouched metrics

